# XlsxConverter 적용 가이드

XlsxConverter는 기획 데이터 엑셀(GameData.xlsx) 하나에서 시트별 JSON, 그 JSON을 읽는 C++/C# 행 코드, enum·상수 정의를 한 번에 만드는 변환 도구입니다. 엑셀 헤더가 스키마이고 코드는 그 스키마에서 나오므로 데이터와 코드가 항상 일치합니다.

## 1. 전체 구조

```
GameData.xlsx ──▶ XlsxConverter ──┬──▶ <시트>.json
(--out)
   데이터 시트 + 패킷 정의 시트      ├──▶ <시트>.gen.h,
Enums/EnumStrings/Consts.gen.h, GeneratedCommon.h   (--cpp-out)
                                  ├──▶ ZonePacketTypes.gen.h,
ZonePacketStructs.gen.h                  (--cpp-out)
                                  ├──▶ <시트>.gen.cs, Enums.gen.cs,
Consts.gen.cs                     (--cs-out)
                                  ├──▶ ProtocolTypes.gen.cs, Packets.gen.cs
(--proto-cs-out)
                                  └──▶ cpp_packet_sizes.txt
(--proto-sizes-out)
```

- 시트 하나가 JSON 파일 하나, C++ 구조체 하나(`S<시트>Row`), C# 클래스 둘(`<시트>Row`, `<시트>Table`)이 됩니다.
- `Enum`, `Const` 시트는 JSON이 아니라 형식 정의입니다. enum과 상수 코드만 만듭니다.
- 패킷 정의 시트 5개(`PacketEnum`, `PacketConst`, `PacketGroup`, `Packet`, `PacketField`)도 JSON이 아니라 형식 정의입니다. 패킷 구조체·번호·결과 코드 enum·프로토콜 상수의 C++ 코드와 C# 코드(`Build` / `Parse` / `ToBytes`), C++ `sizeof` 대조용 크기 목록을 만듭니다. 시트가 없으면 아무것도 만들지 않습니다.
- 코드 생성은 출력 폴더를 지정한 언어만 합니다. `--cpp-out`만 주면 C++, `--cs-out`만 주면 C#, 둘 다 주면 둘 다 만듭니다. 패킷 C#은 `--proto-cs-out`, 크기 목록은 `--proto-sizes-out`으로 따로 지정하며, 패킷 C++ 헤더는 `--cpp-out` 폴더에 함께 생성됩니다.
- 오류가 하나라도 있으면 모든 오류를 출력하고 아무 파일도 쓰지 않습니다.

## 2. 프로젝트 구성

| 파일 | 역할 |
| --- | --- |
| `XlsxConverter.cpp` | 진입점. 명령행 해석, 시트 읽기 순서, 파일 쓰기(원자적 쓰기, 변경된 파일만 쓰기, 낡은 생성 파일 정리), 증분 실행 |

| 파일 | 역할 |
| --- | --- |
| `SheetConverter.h/.cpp` | 시트 → JSON 변환. 헤더 해석 (`Name:type[min..max]?`), 셀 값 검증 |
| `RowSchema.h` | 헤더에서 읽은 스키마, Enum/Const 정의 구조체, 타입 표 (`FindTypeInfo`, `FindBaseInfo`) |
| `DefinitionSheets.h/.cpp` | `Enum`, `Const` 시트, 패킷 정의 시트 5개 해석과 검증 |
| `RowGenerator.h/.cpp` | 스키마 → C++/C# 코드 생성 |
| `PacketSchema.h` | 패킷 정의 시트에서 읽은 내용(상수, 번호 그룹, 패킷, 필드)과 계산된 패킷 크기 |
| `PacketGenerator.h/.cpp` | 패킷 스키마 → C++ 구조체·enum·번호, C# Build/Parse/ToBytes, 크기 목록 |
| `CodeGenCommon.h` | RowGenerator와 PacketGenerator가 함께 쓰는 C# 예약어 목록, 줄 끝 주석 접미사 |
| `ValueFormat.h/.cpp` | 숫자·문자열 표기 규칙(JSON 출력용) |
| `Utf8Literal.h` | `U8("...")` 매크로. 한글 리터럴을 항상 UTF-8 바이트로 만듦 |

## 타입 표

`RowSchema.h`의 표 하나가 기본 타입과 enum 기반형의 모든 매핑을 가집니다. 타입을 추가하거나 바꿀 때는 이 표만 고칩니다.

| 이름 | C++ 멤버 | C++ 상수 | C# | rapidjson | 정수 범위 |
| --- | --- | --- | --- | --- | --- |
| `int` | `int32` | `std::int32_t` | `int` | `GetInt()` | -2147483648 .. 2147483647 |
| `uint` | `uint32` | `std::uint32_t` | `uint` | `GetUint()` | 0 .. 4294967295 |
| `float` | `double` | `double` | `double` | `GetDouble()` | 해당 없음 |
| `string` | `_tstring` | `const TCHAR*` | `string` | `GetString()` | 해당 없음 |
| `bool` | `bool` | `bool` | `bool` | `GetBool()` | 해당 없음 |

enum 기반형(`Base` 열)은 `uint8`, `uint16`, `uint32`, `int32`이고 C++에서는 `std::uintN_t` / `std::int32_t`, C#에서는 `byte` / `ushort` / `uint` / `int`가 됩니다.

# 3. 엑셀 작성 규칙

## 시트와 행

- 시트 이름이 곧 `<시트>.json` 파일 이름, JSON 최상위 키, C++ 구조체 이름( `S<시트>Row` )이 됩니다. 예) `Monster` 시트 → `Monster.json` = `{"Monster":[...]}`
- 1행은 헤더, 2행부터 데이터입니다.
- 첫 칸이 `#` 로 시작하는 행과 완전히 빈 행은 건너뜁니다.
- 이름이 `_` 로 시작하는 시트(예: `_README` )는 변환하지 않습니다. ( `--sheets` 로 직접 지정하면 변환합니다.)
- 예약 시트 이름 `Enums` , `Consts` , `EnumStrings` , `GeneratedCommon` 은 생성 파일 이름과 겹치므로 시트 이름으로 쓸 수 없습니다.

## 헤더 문법

`필드명:타입[min..max]?`

| 요소 | 설명 |
| --- | --- |
| 필드명 | JSON 키이자 C++/C# 멤버 이름의 원본. 영문, 숫자, `_` 만, 숫자로 시작 불가. C++/C# 키워드는 쓸 수 없고 `kFile` , `kTable` , `Specs` , `ToJSON` , `FromJSON` , `enumOk` 도 예약되어 있음 |
| 타입 | `int` , `uint` , `float` , `string` , `bool` , 또는 `Enum` 시트에 정의한 enum 이름 (예: `Shape:ESkillShape` ) |
| `[min..max]` | `int` / `uint` 의 허용 범위. 한쪽 생략 가능. 예) `uint[1..]` , `int[-1000..1000]` , `uint[..100]` . enum에는 쓸 수 없음 |
| `?` | 선택 열. 빈 칸이면 오류 없이 JSON 필드를 생략하고, C++은 `HasMember` 로 확인해 기본값을 둠 |

범위를 쓰지 않아도 값은 C++ 멤버 타입( `int32` / `uint32` )에 담기는 범위여야 합니다.

## 셀 값 규칙

| 타입 | 규칙 |
| --- | --- |
| `int` , `uint` | 정수여야 함. 소수는 오류. `uint` 는 음수 불가. 헤더 범위 또는 타입 범위를 벗어나면 오류 |
| `float` | 숫자 |

| 타입 | 규칙 |
| --- | --- |
| `string` | 빈 칸은 오류 ( `?` 가 있으면 필드 생략) |
| `bool` | `TRUE` / `FALSE` , `1` / `0` , `yes` / `no` , `y` / `n` |
| enum | 멤버 이름이 정확히 일치해야 함. 오타는 허용 목록과 함께 오류 |

## Enum 시트

열: `Enum` , `Name` , `Value` (선택), `Base` (선택), `Comment` (선택)

- 같은 `Enum` 이름의 행들이 한 enum이 되고 순서는 시트의 행 순서입니다.
- `Value` 를 비우면 앞 값 + 1이고 첫 값은 0입니다.
- `Base` 는 `uint8` (기본), `uint16` , `uint32` , `int32` 중 하나이며 enum의 첫 행에만 써도 됩니다. 값은 기반형 범위 안이어야 합니다.
- 멤버 이름 중복, 값 중복, C++ 키워드 이름은 오류입니다.
- EnumComment 열(enum 설명)은 PacketEnum 시트에만 있고 enum의 첫 행에만 쓸 수 있습니다. 일반 Enum 시트에는 이 열이 없습니다.

## Const 시트

열: `Name` , `Type` ( `int` , `uint` , `float` , `string` , `bool` ), `Value` , `Comment` (선택)

- Name은 식별자여야 하고 중복될 수 없습니다. 값은 Type에 맞아야 합니다.

## 패킷 정의 시트

패킷 프로토콜은 시트 5개로 정의합니다. 이 시트들은 JSON으로 변환되지 않고 코드 생성에만 쓰이며, 시트 이름은 예약어입니다.

| 시트 | 내용 | 열 |
| --- | --- | --- |
| `PacketEnum` | 결과 코드 enum. `Enum` 시트와 같은 규칙 | `Enum` , `Name` , `Value` , `Base` , `Comment` , `EnumComment` |
| `PacketConst` | 프로토콜 상수 ( `Type` : `uint16` , `uint32` , `size` ). `Comment` 가 비면 앞 상수의 설명에 이어 쓰임 | `Name` , `Type` , `Value` , `Comment` |
| `PacketGroup` | 패킷 번호 구간과 그 구간의 번호 열거형 | `Group` , `Enum` (C++ 열거형 이름), `CsEnum` , `Min` , `Max` , `Section` (구조체 파일의 구분 주석), `Comment` |

| 시트 | 내용 | 열 |
| --- | --- | --- |
| `Packet` | 패킷 목록 | `Name`, `Group`, `Id`, `Cs`, `Summary`, `Comment`(구조체 `@brief`), `Detail`(`@details`) |
| `PacketField` | 패킷 본문의 필드. 시트 순서가 와이어 순서 | `Packet`, `Name`, `Type`, `Len`, `Enum`, `Comment` |

- 헤더(`size`, `type` 4바이트)는 자동으로 붙습니다. 구조체는 `#pragma pack(1)`이고 크기는 `sizeof` `static_assert`로 고정되므로, 시트와 구조체가 어긋나면 컴파일이 멈춥니다.
- 필드 `Type`: `uint8`, `bool8`, `uint16`, `uint32`, `int32`, `uint64`, `float`, `char`. `bool8`은 C++에서 `uint8`, C#에서 `bool`입니다. `char`는 `Len`(숫자 또는 `PacketConst` 이름)이 필요하고 `Cs=None` 패킷에서만 쓸 수 있습니다.
- `Enum` 열은 필드 값의 의미를 나타내는 `PacketEnum`의 enum입니다. 필드 타입과 enum의 기반 형이 같아야 합니다. C++ 멤버는 정수형 그대로이고 C#은 enum 타입이 됩니다.
- `Cs` 열: `Send`(클라이언트가 보냄, `Build` 생성), `Recv`(클라이언트가 받음, `Parse`/`ToBytes` 생성), `Both`, `None`(기본값, 서버 간 패킷).
- `Id`는 그룹의 `Min`..`Max` 안에 있어야 하고 전체에서 유일해야 합니다. 패킷을 추가할 때는 `Packet` 시트에 행을, `PacketField` 시트에 그 패킷의 필드 행을 추가하고 다시 실행하면 됩니다.

## 4. 명령행

```
XlsxConverter64.exe <file.xlsx> --out <dir> [--cpp-out <dir>] [--cs-out
<dir>] [--cs-namespace <name>]
                    [--proto-cs-out <dir>] [--proto-cs-namespace <name>] [--
proto-sizes-out <file>]
                    [--sheets Name ...] [--incremental]
```

| 옵션 | 설명 |
| --- | --- |
| `--out` | JSON 출력 폴더 (필수) |
| `--cpp-out` | C++ 헤더 출력 폴더 |
| `--cs-out` | C# 출력 폴더 |
| `--cs-namespace` | C# 네임스페이스 (기본 `GameDataRows`) |
| `--sheets` | 일부 시트만 JSON 변환. 코드 생성과 함께 쓸 수 없음 (일부 시트만으로 만든 헤더는 다른 구조체를 지우기 때문) |

| 옵션 | 설명 |
|---|---|
| `--incremental` | 엑셀의 수정 시각/크기가 마지막 실행 때와 같고 출력 파일이 모두 있으면 엑셀을 열지 않고 종료 |
| --proto-cs-out | 패킷 프로토콜 C# 출력 폴더. --cs-out과 다른 폴더여야 함 (3절 패킷 정의 시트) |
| --proto-cs-namespace | 패킷 C# 네임스페이스 (기본은 --cs-namespace 값) |
| --proto-sizes-out | "<구조체> <바이트>" 목록 파일. C# 테스트가 C++ sizeof와 대조하는 입력. 폴더 없이 파일 이름만 줘도 현재 폴더에 만듦 |

종료 코드: `0` 성공, `1` 변환 실패(데이터 오류), `2` 사용법/파일 오류

### 파일 쓰기 규칙

- 생성 파일은 **BOM 있는 UTF-8**, JSON은 BOM 없는 UTF-8입니다.
- 내용이 같은 파일은 다시 쓰지 않습니다(줄바꿈 차이는 무시). 불필요한 재빌드를 막습니다.
- 파일은 임시 파일에 쓴 뒤 교체하므로 읽는 쪽이 반쯤 쓰인 파일을 보지 않습니다.
- 이번에 만들지 않은 `*.gen.h` / `*.gen.cs` 중 상단에 "자동 생성 파일" 표식이 있는 것만 지웁니다. 손으로 만든 파일은 건드리지 않습니다.

### 증분 실행

- 상태는 `--out` 폴더의 `.XlsxConverter.stamp`에 저장합니다. 버전 관리에서 제외하세요.
- 상태에는 실행 인자, 실행 파일의 수정 시각, 엑셀의 수정 시각/크기, 만들어진 출력 파일 목록을 기록하고, 모든 출력이 끝난 뒤 마지막에 씁니다.
- 인자가 바뀌었거나 도구를 다시 빌드했거나 엑셀이 달라졌거나 출력 파일이 하나라도 없으면 다시 변환합니다.
- 엑셀을 읽을 수 없으면(Excel에 열려 있는 경우 등) 같은 인자로 만든 출력이 있을 때 **경고만 내고 종료 코드 0**으로 끝나 기존 출력으로 빌드를 계속합니다.

## 5. 생성되는 C++ 코드

| 파일 | 내용 |
|---|---|
| `<시트>.gen.h` | `GameDataRows::S<시트>Row` 구조체 |
| `Enums.gen.h` | `GameDataEnums`의 enum class들. 다른 헤더에 의존하지 않음 |

| 파일 | 내용 |
| --- | --- |
| `EnumStrings.gen.h` | `ToString(enum)`, `TryParse(const TCHAR*, enum&)` |
| `Consts.gen.h` | `GameDataConsts::k<Name>` constexpr 상수 |
| `ZonePacketTypes.gen.h` | 패킷 프로토콜 상수(PacketConst), enum class(PacketEnum), 패킷 번호 열거형(PacketGroup + Packet). --cpp-out 폴더 |
| `ZonePacketStructs.gen.h` | <이름>Packet : PacketHeader 구조체와 sizeof static_assert (Packet + PacketField) |
| `GeneratedCommon.h` | 위 파일들을 한곳에서 include하는 대표 헤더 (정의 헤더 먼저) |

## 행 구조체

```
struct SHeroRow
{
    uint32   id = 0;
    _tstring name;
    int32    maxHp = 0;
    // ...

    static constexpr const TCHAR* kFile  = _T("Hero.json");
    static constexpr const TCHAR* kTable = _T("Hero");

    static std::initializer_list<SFieldSpec> Specs();   // 필드 이름, 종류, 범위, 선택 여부
    void ToJSON(_tValue& value, _tDocument::AllocatorType& allocator) const;
    void FromJSON(const _tValue& value);
};
```

- 멤버 이름은 필드명의 첫 글자를 소문자로 한 이름이고, 멤버 접근은 `this->`로 합니다.
- `Specs()`는 헤더의 타입과 `[min..max]`를 그대로 담아 로더가 범위를 검증하는 데 씁니다.
- 선택 열은 `Specs`에 선택 표시가 붙고 `FromJSON`에서 `HasMember`로 확인합니다.
- enum 필드가 있는 시트는 `enumOk` 멤버를 가집니다. JSON의 멤버 이름이 `TryParse`로 변환되지 않으면 `false`입니다.

### 로더에서 쓰는 방법

```
#include "../GameCommon/Generated/GeneratedCommon.h"   // 시트별 헤더를 하나씩 include하지 않는다
using namespace GameDataRows;
```

- 로더는 `S<시트>Row::kFile`, `Specs()`, `FromJSON()`을 사용하는 `ReadTable`로 JSON을 읽고, enum 필드가 있는 시트는 `enumOk`를 확인합니다.
- `SkillData.h`는 `using GameDataEnums::ESkillShape;`처럼 생성된 enum을 이름만 가져와 기존 코드가 그대로 동작합니다. 상수는 `GameDataConsts::k<Name>`을 참조합니다.
- 로더는 생성된 enum과 서버의 enum이 같은 타입인지 `static_assert`로 확인합니다.
- `BlockMapFile.cpp`는 `Block.gen.h`의 `SBlockRow`를 사용합니다.

### 패킷 헤더

- `ZonePacketStructs.gen.h`의 구조체는 `#pragma pack(1)`이고 `PacketHeader`(4바이트) 다음에 `PacketField` 시트 순서대로 필드가 이어집니다. `sizeof`는 `static_assert`로 고정되어 시트와 어긋나면 컴파일이 멈춥니다.
- enum 열이 있는 필드도 C++ 멤버는 `uint8` 같은 정수형 그대로이고 enum 이름은 주석으로 남습니다. `bool8`도 `uint8`입니다.
- 기존 `ZonePacketTypes.h`는 생성 헤더를 include만 하는 전달 헤더가 되고, `ZonePacket.h`는 구조체를 생성 헤더에서 가져오며 프레이밍·중계 패킷 조립 같은 손으로 쓴 도우미만 남깁니다. 둘 다 `../Generated/...` 상대 경로로 include하므로 서버와 Gateway의 `#include "ZonePacket.h"`는 그대로이고, `GameCommon\Generated`가 `GameCommon\ZoneCommon`과 같은 부모 아래에 있어야 합니다.
- 가변 길이 중계 패킷(`ZoneRelayToZone` 등)은 고정 부분만 생성됩니다. 뒤따르는 본문을 조립하는 함수는 `ZonePacket.h`에 손으로 남아 있습니다.

## 6. 생성되는 C# 코드

| 파일 | 내용 |
|---|---|
| `<시트>.gen.cs` | `<시트>Row`(`FileName`, `TableKey`, 속성, `Validate`)와 `<시트>Table`(JSON 최상위 객체) |
| `Enums.gen.cs` | enum들 |
| `Consts.gen.cs` | `GameConsts`의 `public const` 상수 |
| `ProtocolTypes.gen.cs` | ProtocolConst(partial), enum, 패킷 번호 열거형. --proto-cs-out 폴더 |

| 파일 | 내용 |
| --- | --- |
| `Packets.gen.cs` | Build.<패킷>(...) 송신, <패킷>.Parse(byte[]) 수신, <패킷>.ToBytes() 서버 쪽 레이아웃 직렬화 |

- 속성 이름이 JSON 키와 같아 별도 특성이 필요 없습니다.
- `Validate(where, errors)` 는 헤더의 `[min..max]` 를 벗어난 값을 `errors` 에 덧붙입니다. `<시트>Table.Validate(errors)` 는 모든 행을 검사합니다.
- JSON에는 enum이 멤버 이름 문자열로 저장되므로 역직렬화에 문자열 enum 변환기가 필요합니다. ( `System.Text.Json` 의 `JsonStringEnumConverter` 또는 Newtonsoft의 `StringEnumConverter` )
- enum 이름이 `<시트>Row` / `<시트>Table` 이름과 겹치면 생성 단계에서 오류입니다.

### 패킷 코드

- `Cs` 열이 `Send` / `Both` 인 패킷은 `Build.<패킷>(필드...)` 가 되고 크기 상수 `Build.<패킷>Size` 가 함께 생깁니다. `Recv` / `Both` 인 패킷은 필드를 담는 클래스와 `Parse(byte[])` , `ToBytes()` 가 됩니다. `ToBytes()` 는 테스트의 가짜 서버와 오프라인 데모가 씁니다.
- 이름 규칙: enum의 앞 `E` 와 상수의 앞 `k` 는 떼고( `EGatewayResult` → `GatewayResult` , `kMaxClientPacketBytes` → `MaxClientPacketBytes` ), 필드 이름은 첫 글자만 대문자로 합니다( `charId` → `CharId` ). `bool8` 은 `bool` , `Enum` 열이 있는 필드는 그 enum 타입이 됩니다.
- `ProtocolConst` 는 `partial` 이라 시트에 없는 값은 손으로 쓴 `PacketTypes.cs` 가 같은 클래스에 더합니다. ( `HeaderSize` , 시트에 없는 `EntityType` )
- 네임스페이스는 `--proto-cs-namespace` (기본은 `--cs-namespace` 값)입니다. `char` 배열은 C# 생성을 지원하지 않으므로 해당 패킷은 `Cs=None` 이어야 합니다.
- `--proto-sizes-out` 으로 만든 `cpp_packet_sizes.txt` 를 뷰어 테스트가 읽어 C# 클래스의 `Size` 와 C++ `sizeof` 를 대조합니다.

## 7. 인코딩

- 생성 파일은 BOM 있는 UTF-8(줄바꿈 LF), JSON은 BOM 없는 UTF-8입니다.
- XlsxConverter 소스( `.cpp` / `.h` )는 **BOM 있는 UTF-8**로 저장합니다.
- MSVC는 `/utf-8` 이 없으면 일반 문자열 리터럴을 시스템 코드 페이지(한국어 Windows는 CP949)로 만듭니다. 그대로 파일에 쓰면 한글 주석이 깨지므로, 한글이 든 모든 리터럴은 `U8("...")` 로 감쌉니다. 컴파일러 옵션과 무관하게 항상 UTF-8 바이트가 나옵니다. 새 한글 문자열을 추가할 때도 `U8()` 를 씁니다.
- 엑셀 셀에서 읽은 문자열은 xInt가 UTF-8로 주므로 영향이 없고, 주석도 영향이 없습니다.
- 프로젝트에 `/utf-8` 을 추가하면 일반 리터럴도 안전해집니다.

# 8. 빌드 자동화

## Pre-Build Event

서버 빌드 시 엑셀 수정이 자동 반영되도록 ZoneServer 프로젝트에 설정합니다.

1. 솔루션 탐색기에서 ZoneServer 프로젝트를 우클릭하고 **속성**을 엽니다.
2. 위쪽 **구성**을 `모든 구성`, **플랫폼**을 `모든 플랫폼`으로 바꿉니다.
3. **빌드 이벤트 → 빌드 전 이벤트**의 **명령줄**에 아래를 넣습니다.

```
"$(SolutionDir)Bin\x64\Release\XlsxConverter64.exe"
"$(SolutionDir)GameData\GameData.xlsx" --out "$(SolutionDir)GameData" --cpp-
out "$(SolutionDir)GameCommon\Generated" --cs-out
"$(SolutionDir)GameCommon\Generated\CSharp" --cs-namespace Game.Data --proto-
cs-out "$(SolutionDir)GameViewer\Core\Protocol\Generated" --proto-cs-
namespace GameViewer.Core.Protocol --proto-sizes-out
"$(SolutionDir)GameViewer\Tests\cpp_packet_sizes.txt" --incremental
```

경로는 실제 폴더 구조에 맞게 고칩니다. `$(SolutionDir)`은 끝에 `\`가 포함되어 있습니다.

`--proto-*` 옵션은 패킷 정의 시트가 있는 엑셀에서만 쓰고, 뷰어 폴더가 솔루션 하위가 아니면 경로를 실제 위치에 맞춥니다. `--proto-cs-out`은 `--cs-out`과 다른 폴더여야 합니다.

| 상황 | 동작 |
|---|---|
| 엑셀 변경 없음 | `up to date`로 엑셀을 열지 않고 종료, 빌드 계속 |
| 엑셀 변경됨 | 다시 변환. 내용이 같은 생성 파일은 다시 쓰지 않아 재빌드가 늘지 않음 |
| 데이터 오류 | 종료 코드 1로 빌드 실패. 오류 목록이 출력 창에 나오고 파일은 쓰지 않음 |
| 엑셀이 열려 읽을 수 없음 | 같은 인자로 만든 출력이 있으면 경고 후 종료 코드 0 |

## 프로젝트 종속성 (빌드 순서)

Pre-Build가 쓰는 `XlsxConverter64.exe`가 ZoneServer보다 먼저 빌드되게 합니다.

1. XlsxConverter 프로젝트가 솔루션에 없으면 솔루션을 우클릭해 **추가 → 기존 프로젝트**로 넣습니다.
2. ZoneServer 프로젝트를 우클릭하고 **빌드 종속성 → 프로젝트 종속성...**을 선택합니다.
3. **종속성** 탭에서 프로젝트 드롭다운을 `ZoneServer`로 두고 목록의 **XlsxConverter**에 체크합니다.
4. **빌드 순서** 탭에서 XlsxConverter가 ZoneServer보다 위에 있는지 확인합니다.

주의할 점은 세 가지입니다.

- exe 출력 폴더와 Pre-Build 명령줄의 경로가 같아야 합니다.
- Debug로 빌드하면 XlsxConverter도 Debug로 만들어집니다. 명령줄의 `Release`를 `$(Configuration)`으로 바꾸면 구성에 따라갑니다.
- 솔루션 **구성 관리자**에서 ZoneServer가 빌드되는 구성에 XlsxConverter의 **빌드**가 켜져 있어야 합니다.

종속성을 걸지 않는다면 XlsxConverter를 미리 한 번 빌드해 두고, 소스를 고칠 때마다 직접 다시 빌드합니다.

### 생성 헤더를 탐색기에 표시 (선택)

생성 헤더는 프로젝트 항목으로 등록하지 않아도 `#include` 경로만 맞으면 컴파일됩니다. 시트를 추가하면 XlsxConverter가 `<시트>.gen.h`를 만들고 `GeneratedCommon.h`에 include 줄을 넣으므로, 로더 쪽에서 고칠 것은 없습니다. 헤더가 바뀌면 MSBuild가 include 의존성을 추적해 해당 소스를 다시 컴파일합니다.

솔루션 탐색기에 보이게 하려면 `Generated` 필터를 만들고 생성 헤더를 항목으로 추가합니다. C++ 프로젝트(`.vcxproj`)는 `*.gen.h` 같은 와일드카드 항목을 지원하지 않습니다. 탐색기에 나타나지 않고, VS가 프로젝트를 저장할 때 문제가 생길 수 있으므로 파일마다 항목을 하나씩 둡니다.

`ZoneServer.vcxproj`에는 헤더마다 항목 하나를 둡니다.

```
<ItemGroup>
  <ClInclude Include="..\GameCommon\Generated\GeneratedCommon.h" />
  <ClInclude Include="..\GameCommon\Generated\Hero.gen.h" />
  <ClInclude Include="..\GameCommon\Generated\Monster.gen.h" />
  <!-- 생성된 *.gen.h마다 한 줄씩 -->
</ItemGroup>
```

탐색기에서 어느 폴더에 보일지는 `ZoneServer.vcxproj.filters`가 정합니다. 필터를 한 번 정의하고, 위의 항목마다 `<Filter>`를 붙입니다. 특히 `<Link>` 메타데이터는 C++ 프로젝트가 사용하지 않으므로 쓰지 않습니다.

```
<ItemGroup>
  <Filter Include="Generated">
    <UniqueIdentifier>{새 GUID}</UniqueIdentifier>
  </Filter>
</ItemGroup>
<ItemGroup>
  <ClInclude Include="..\GameCommon\Generated\GeneratedCommon.h">
    <Filter>Generated</Filter>
  </ClInclude>
  <ClInclude Include="..\GameCommon\Generated\Hero.gen.h">
    <Filter>Generated</Filter>
  </ClInclude>
  <ClInclude Include="..\GameCommon\Generated\Monster.gen.h">
    <Filter>Generated</Filter>
  </ClInclude>
</ItemGroup>
```

두 파일을 손으로 고치기보다 VS에서 하는 편이 안전합니다. 프로젝트에서 새 필터 `Generated`를 만들고, 필터를 우클릭해 추가 → 기존 항목으로 `GeneratedCommon.h`와 `*.gen.h`를 모두 고르면 `.vcxproj`와 `.vcxproj.filters`가 함께 갱신됩니다. 필터를 만들지 않고 추가하면 `.vcxproj.filters`에 `<Filter>` 줄이 생기지 않으므로 탐색기에서 원하는 폴더에 보이지 않습니다.

- 경로는 `.vcxproj` 위치 기준으로 고칩니다.
- 새 시트를 추가하면 생기는 `<시트>.gen.h`는 같은 방법으로 한 번 더 추가해야 탐색기에 보입니다. 컴파일에는 영향이 없습니다.
- 생성 파일이므로 직접 고치지 않습니다. 프로젝트에서 제외해도 다음 실행 때 다시 만들어지고 컴파일에는 영향이 없습니다.
- XlsxConverter가 `.vcxproj`를 직접 수정하는 방식은 쓰지 않습니다. 실행 중인 VS와 충돌하고 팀원의 프로젝트 파일 수정과 병합 문제가 생길 수 있기 때문입니다.

## 9. 도입 절차

1. XlsxConverter 프로젝트를 Windows에서 빌드합니다. (xlnt 필요)
2. 회귀 테스트를 실행해 모두 통과하는지 확인합니다. (10절)
3. `XlsxConverter.bat`을 실행해 JSON과 생성 파일을 만들고 생성 파일의 한글 주석이 정상인지 확인합니다.
4. 생성 헤더는 프로젝트에 등록하지 않아도 빌드됩니다. 솔루션 탐색기에 표시하려면 8절의 방법으로 `Generated` 필터에 추가합니다. (선택)
5. 로더는 `GeneratedCommon.h` 하나만 include합니다.
6. 솔루션을 빌드합니다.
7. (선택) 8절의 Pre-Build Event와 프로젝트 종속성을 설정합니다.

패킷 프로토콜 생성을 쓰는 경우는 위 절차에 이어 다음을 합니다.

1. 새 소스 파일 4개( `PacketGenerator.h` , `PacketGenerator.cpp` , `PacketSchema.h` , `CodeGenCommon.h` )를 이 프로젝트에 추가합니다. `CodeGenCommon.h` 는 `RowGenerator.cpp` 도 사용합니다.
2. `GameCommon\ZoneCommon\ZonePacketTypes.h` 와 `ZonePacket.h` 를 전달/도우미만 남긴 버전으로 교체합니다.
3. 뷰어의 `Core\Protocol\PacketTypes.cs` 를 축소본으로 교체하고 `Packets.cs` 를 삭제합니다.
4. `Core\Protocol\Generated\` 폴더와 새 `Tests\cpp_packet_sizes.txt` 를 넣습니다.
5. 서버 코드의 `#include` 는 바꾸지 않습니다.

## 10. 회귀 테스트

`XlsxConverter.Tests` 는 골든 파일 비교 테스트입니다. 케이스마다 `sheets\*.tsv` 로 `.xlsx` 를 만들어 실제 XlsxConverter를 실행하고, 결과를 `expect\` 의 정답 파일과 **바이트 단위**로 비교합니다.

```
pip install openpyxl
cd XlsxConverter.Tests
python run_tests.py --exe ..\Bin\x64\Release\XlsxConverter64.exe
```

| 케이스 | 확인 내용 |
|---|---|
| `basic` | 샘플 GameData.xlsx 전체 (JSON, C++, C#) |
| `enum_const` | enum 자동 증가값, uint32 기반 2^31 이상 값, 선택 enum 열, 모든 Const 타입, 문자열 이스케이프 |
| `err_header` | 헤더 문법 오류들 |
| `err_enum_const` | Enum/Const 시트 오류들 |
| `err_data` | 범위, enum 오타, 정수 아님, bool 아님, 빈 칸 |
| `err_reserved` | 예약 시트 이름 |
| `err_cpp_keyword` | C++ 키워드를 enum/멤버 이름으로 쓴 경우 |
| `packets` | 패킷 시트 5개: C++ 헤더 2개, C# 2개, 크기 목록 (Send/Recv/Both/None, bool8, enum 필드, char 배열, 구간) |
| `err_packet` | 패킷 시트 오류들: 상수 타입·범위, 구간 겹침, Id 중복·구간 밖, 알 수 없는 Group/Cs/Type, enum 기반형 불일치, char Len, C# 미지원 char |

| 케이스 | 확인 내용 |
| --- | --- |
| `scn/*` | BOM 유무, 같은 내용은 다시 쓰지 않음, 낡은 생성 파일 정리, `--incremental` 동작 |

- 실패 케이스는 종료 코드 1, `expect_stderr.txt`의 문장이 stderr에 모두 있음, 출력 파일이 하나도 없음을 확인합니다.
- 생성 코드 형식을 바꾼 경우에만 `--update`로 정답 파일을 갱신하고, 바뀐 내용을 diff로 검토합니다. 오류 메시지를 바꾸면 `expect_stderr.txt`도 함께 고칩니다.

## 11. 문제 해결

| 증상 | 원인과 조치 |
| --- | --- |
| 생성 파일의 한글 주석이 깨짐 | 한글 리터럴에서 `U8()` 누락. 해당 리터럴을 감싸고 다시 빌드해 실행 |
| C2440 `GameDataEnums::X` → `X` | 서버 쪽 enum 정의가 생성 enum과 다른 타입. `SkillData.h`처럼 `using GameDataEnums::X;`로 가져오게 함 |
| 종료 코드 1과 오류 목록 | 엑셀 데이터 오류. `시트!행 열` 위치를 고침 |
| 종료 코드 2 | 인자 또는 파일 오류. 경로와 인자 확인 |
| 엑셀을 고쳤는데 `up to date` | 인자가 같고 엑셀 수정 시각/크기가 같은 경우. `--incremental`을 빼고 실행해 강제 변환 |
| `--cpp-out`과 `--sheets` 오류 | 함께 쓸 수 없음. 코드 생성은 모든 시트로 해야 함 |
| --proto-* 사용 시 "패킷 정의 시트가 없습니다" | 엑셀에 PacketEnum, PacketConst, PacketGroup, Packet, PacketField 시트가 없음. 시트를 추가하거나 --proto-* 옵션을 뺍니다 |
| Packet!행 오류: Id 범위 밖, Id가 같음 | PacketGroup의 Min/Max 구간과 Packet의 Id를 확인. 패킷 번호는 전체에서 유일해야 함 |
| C2338 …Packet: wire size differs from the Packet sheet | 생성 헤더를 손으로 고친 경우이거나 오래된 생성물. 엑셀을 고치고 XlsxConverter를 다시 실행 |
| --proto-cs-out 오류: --cs-out과 다른 폴더여야 함 | C# 생성물 두 묶음이 같은 폴더에 섞이면 서로의 *.gen.cs를 지움. 폴더를 분리 |
| 테스트에서 차이가 나옴 | 골든 파일은 xlnt 없는 환경에서 TSV 스텁으로 만든 것. xlnt의 셀 값 해석 차이(숫자 표기, 빈 셀)인지 diff로 확인 |

| 증상 | 원인과 조치 |
| --- | --- |
| 서버 시작 시 game data does not fit the zone ... spawn center of monster N is inside an obstacle | 변환 도구 오류가 아니라 서버의 시작 검증. Spawn 시트의 해당 몬스터 스폰 중심 좌표가 Block 맵의 막힌 칸 안에 있음. 좌표를 막히지 않은 칸으로 옮기고 다시 변환 |

## 알려진 제한 사항

- 가변 길이 중계 패킷(ZoneRelayToZone 등)은 고정 부분만 생성됩니다. 뒤따르는 본문을 조립하는 함수는 ZonePacket.h에 손으로 둡니다.
- char 배열 필드는 C# 생성을 지원하지 않으므로 해당 패킷은 Cs=None(서버 간 패킷)이어야 합니다.
- --sheets는 코드 생성 옵션과 함께 쓸 수 없습니다. 코드 생성은 항상 모든 시트로 합니다.
- --proto-cs-out은 --cs-out과 다른 폴더여야 합니다. 같은 폴더에 두면 서로의 *.gen.cs를 지웁니다.
- Spawn 좌표와 Block 맵의 충돌 같은 시트 사이의 내용 검증은 변환 도구가 아니라 서버가 시작할 때 합니다.